The University of Jordan

King Abdullah II School of Information Technology

Department of Computer Science

CyberSim

A Dual-Perspective Cybersecurity Training Platform

Graduation Project — 2025/2026

Submitted in partial fulfillment of the requirements

for the Bachelor's Degree in Computer Science

Academic Year 2025 – 2026

ABSTRACT

CyberSim is a browser-based cybersecurity training platform designed for university students. It provides a dual-perspective learning environment where students practice Red Team offensive techniques against isolated Docker scenario containers and simultaneously observe the Blue Team defensive telemetry generated by their actions. The platform integrates a Kali-style PTY terminal, a real-time SIEM event feed, structured methodology gating, a Socratic AI tutor, mission scoring, and post-session debrief reports. Three training scenarios are included in the MVP scope: SC-01 NovaMed (web application penetration testing), SC-02 Nexora (Active Directory compromise), and SC-03 Orion (phishing campaign simulation and SOC response). The entire stack is packaged as a single-node Docker Compose deployment, making it portable for university labs and graduation demonstrations. Instructor analytics provide session monitoring, student metrics, and report export capabilities.

Keywords: *cybersecurity training, penetration testing, SOC analysis, Docker, xterm.js, FastAPI, React, Elasticsearch, Socratic AI, dual-perspective*

TABLE OF CONTENTS

LIST OF FIGURES

No table of figures entries found.

LIST OF TABLES

No table of figures entries found.

CHAPTER 1

INTRODUCTION

1.1 Background

Cybersecurity education requires both technical practice and operational understanding. Students often learn offensive techniques such as reconnaissance, vulnerability discovery, exploitation methodology, and post-exploitation reasoning separately from defensive analysis topics such as log review, alert triage, incident response, and reporting. This separation can make it difficult for students to understand the cause-and-effect relationship between an attacker action and the evidence observed by a defender.

CyberSim addresses this learning gap through a dual-perspective training platform. It provides a browser-based Red Team workspace where students interact with isolated scenario environments through a Kali-style terminal, and a Blue Team workspace where corresponding telemetry, SIEM events, notes, and response activities are analyzed. The platform is designed for university training, so all offensive learning activities are constrained to deliberately vulnerable Docker containers and internal Docker networks.

1.2 Motivation

The motivation for CyberSim is to make cybersecurity training more realistic, connected, and safe. Traditional exercises may emphasize either attack execution or defensive monitoring, but students need to see both perspectives in one controlled environment. CyberSim makes this relationship visible by linking terminal actions, scenario progress, SIEM telemetry, notes, scoring, AI hints, and debrief reports.

The project is also motivated by classroom needs. Instructors need a way to monitor student progress, evaluate methodology adherence, review evidence, and export grade-ready data. Students need a guided environment where they can practice without accidentally targeting real systems or receiving unsafe instructions.

1.3 Problem Statement

Cybersecurity students lack an integrated training environment that combines:

- Safe offensive practice inside isolated targets.
- Real-time defensive telemetry and SIEM-style investigation.
- Structured methodology guidance.

- Scenario-based learning objectives.
- Instructor visibility and grading support.
- Post-mission debriefs that explain cause and effect.

Existing tools often solve only part of this problem. CyberSim proposes a unified platform that connects Red Team and Blue Team workflows in a single browser-based application.

1.4 Project Aim

The aim of CyberSim is to design and implement a dual-perspective cybersecurity training platform that allows students to practice Red Team and Blue Team workflows safely inside Docker-isolated scenarios while receiving structured guidance, scoring, telemetry, and debrief reports.

1.5 Project Objectives

Table 1.1: Data table

Objective ID	Objective	Success Indicator
OBJ-01	Provide safe scenario-based offensive practice	All scenario targets run in isolated Docker networks
OBJ-02	Provide Blue Team visibility into scenario activity	SIEM events and triage workflows are available in the Blue workspace
OBJ-03	Support learning through guidance	AI hints and structured hint trees provide bounded Socratic support
OBJ-04	Track methodology and progress	Scenario phases, notes, flags, and gates reflect student progress
OBJ-05	Support assessment	Scoring, reports, and instructor analytics produce reviewable evidence
OBJ-06	Support deployment and demonstration	Docker Compose and demo scripts verify readiness

1.6 Project Scope

The active MVP scope includes exactly three scenarios:

Table 1.2: Data table

Scenario	Name	Focus
SC-01	NovaMed Healthcare	Web application penetration testing and OWASP-style findings
SC-02	Nexora Financial	Active Directory compromise simulation and defender correlation
SC-03	Orion Logistics	Phishing campaign simulation and SOC response

Out of scope:

- Testing real external systems.
- Building real malware.
- Allowing scenario containers unrestricted internet access.
- Expanding beyond SC-01 through SC-03 before the MVP is fully verified.

1.7 Target Users and Stakeholders

Table 1.3: Data table

Stakeholder	Need
Student as Red Team operator	Practice structured offensive methodology in a safe environment
Student as Blue Team analyst	Investigate telemetry, triage alerts, and write evidence-driven reports
Instructor	Monitor progress, review sessions, export grades, and identify common weaknesses
System administrator	Deploy, configure, verify, and recover the platform
Examiner	Evaluate technical depth, project completeness, safety, and educational

	contribution

1.8 Software and Hardware Requirements

Software requirements:

- Docker Desktop or Docker Engine with Docker Compose v2.
- Python 3.11 for backend development.
- Node.js 18 or newer for frontend development.
- Modern web browser.
- PostgreSQL, Redis, Elasticsearch, Filebeat, Nginx/Caddy through Docker services.

Hardware requirements:

- Minimum 8 GB RAM for local development.
- Recommended 16 GB RAM for smoother full-stack scenario execution.
- CPU and storage sufficient for Docker images, Elasticsearch, and scenario containers.

1.9 Limitations and Assumptions

CyberSim assumes a local or demo Docker host with enough resources to run the core stack and selected scenario profiles. The platform is optimized for a university demonstration and classroom lab, not for large-scale multi-tenant production without future scaling work.

The AI hint system can operate in fallback mode when the external AI key is unavailable. This keeps the platform usable, but live AI quality depends on valid provider configuration and rate/budget controls.

1.10 Expected Outputs

Expected project outputs include:

- Browser-based CyberSim application.
- Red Team and Blue Team workspaces.
- Three scenario environments.
- Scenario notes, hints, scoring, and reports.
- Instructor dashboard and analytics.
- Docker-based deployment configuration.
- Final graduation report, technical appendices, diagrams, Canva visual report, presentation, and poster.

1.11 Project Schedule and Methodology

CyberSim was developed incrementally through phases covering planning, infrastructure, backend foundation, frontend workspaces, scenario engine, terminal proxy, SIEM, AI monitor, reports, scoring, instructor analytics, readiness, and scenario depth. The documentation package follows the KASIT product-based structure and adds a commercial-grade technical documentation layer.

1.12 Report Outline

Chapter 1 introduces the project, motivation, scope, users, requirements, and expected outputs. Chapter 2 reviews related systems. Chapter 3 defines requirements and analysis. Chapter 4 presents system design. Chapter 5 explains implementation. Chapter 6 presents testing, installation, and user guidance. Chapter 7 concludes the project and proposes future work.

CHAPTER 2

RELATED EXISTING SYSTEMS

2.1 Overview of Cybersecurity Training Paradigms

Cybersecurity education has traditionally relied on segmented training paradigms. Students are taught offensive concepts (reconnaissance, exploitation, privilege escalation) separately from defensive concepts (log analysis, intrusion detection, incident response, and triage). This segmentation prevents students from observing the immediate causal link between an attacker's command and the defender's corresponding telemetry.

Several platforms exist to bridge these training gaps, categorized into offensive platforms, defensive ranges, vulnerable applications, and hybrid systems. This chapter reviews these existing systems and highlights the specific limitations that CyberSim addresses.

2.2 Offensive Training Platforms

2.2.1 TryHackMe

TryHackMe is a cloud-based platform that provides guided rooms covering various cybersecurity topics.

- Strengths: High accessibility via in-browser Kali Linux instances; structured learning paths; gamified scoring.
- Limitations: The platform primarily focuses on the attacker perspective. While some defensive rooms exist, they are isolated from the offensive rooms. Students do not see the real-time telemetry generated by their own offensive actions.
- Relevance: TryHackMe demonstrates the utility of browser-based Kali instances, which inspired CyberSim's containerized Red Team workspace.

2.2.2 Hack The Box (HTB) Academy

Hack The Box provides a highly competitive, practical environment for penetration testing and offensive techniques.

- Strengths: High-fidelity, challenging machine environments; realistic vulnerability chains; structured academy modules.
- Limitations: HTB is predominantly offensive and competitive. It lacks a real-time defensive correlation interface where students can watch logs populate as they run exploit tools. Its difficulty curve can be steep for university students without guided coaching.

- Relevance: HTB's emphasis on realistic exploits highlights the importance of using genuine Kali tools, which CyberSim supports via strict raw Docker PTY execution.

2.2.3 PicoCTF

PicoCTF is an educational Capture The Flag (CTF) platform designed by Carnegie Mellon University for students.

- Strengths: Bounded, gamified, and highly approachable for beginners; covers cryptography, web exploitation, and reverse engineering.
- Limitations: CTF challenges are static and task-oriented rather than methodology-oriented. They do not simulate realistic corporate networks or provide defensive SIEM analysis.
- Relevance: PicoCTF demonstrates that bounded, approachable challenges are effective for student engagement.

2.3 Defensive Training Ranges

2.3.1 CyberDefenders

CyberDefenders is a blue-team-focused training platform that provides CTF-style challenges based on pre-captured PCAP files, memory dumps, and disk images.

- Strengths: Focuses on realistic incident response, threat hunting, and forensics.
- Limitations: Challenges are retrospective and static. Students analyze historical artifacts rather than investigating live attacks as they happen. There is no active offensive component.
- Relevance: CyberDefenders highlights the academic value of structured incident analysis and triage, which CyberSim integrates into its Blue Team workspace.

2.3.2 Splunk Boss of the SOC (BOTS)

Splunk BOTS is a blue-team competition where analysts search through large volumes of real corporate telemetry to investigate a simulated intrusion.

- Strengths: High-fidelity corporate log datasets; realistic multi-stage attack scenarios.
- Limitations: BOTS is entirely retrospective. The attack has already occurred, and students only query the historical index. It requires a heavy commercial SIEM setup (Splunk) and lacks a live offensive interaction panel.
- Relevance: BOTS shows the educational power of querying real SIEM events to reconstruct an attack timeline, which inspired CyberSim's debrief timeline.

2.4 Vulnerable Applications and Labs

2.4.1 OWASP Juice Shop & Damn Vulnerable Web Application (DVWA)

These are intentionally vulnerable web applications designed for security testing practice.

- Strengths: Lightweight; easy to deploy locally; excellent for OWASP Top 10 learning.
- Limitations: They lack automated telemetry generation, SIEM integration, and structured student progress tracking. There is no guidance layer to help students when they get stuck.
- Relevance: These applications serve as the baseline for CyberSim's SC-01 NovaMed scenario, which containerizes a vulnerable PHP/Apache service backed by a ModSecurity WAF.

2.4.2 Metasploitable

Metasploitable is an intentionally vulnerable virtual machine containing numerous security flaws across standard services.

- Strengths: Provides a target for a wide variety of offensive tools (Nmap, Metasploit, Hydra).
- Limitations: It lacks defensive monitoring interfaces, progress tracking, and AI tutoring. It also requires virtual machine virtualization, which is heavier than containerization.
- Relevance: Metasploitable shows the utility of multi-service vulnerable targets, which CyberSim implements using isolated Docker compose profiles.

2.5 Limitations of Existing Systems

While existing systems are highly effective within their specific domains, they exhibit several limitations in the context of university education:

1. Perspective Siloing: Students practice either offensive tools or defensive analysis. They rarely experience the immediate cause-and-effect relationship between a specific command (e.g., sqlmap execution) and its defensive signature (e.g., WAF SQLi alerts in a SIEM).
2. Lack of Methodology Gating: Existing platforms allow students to run advanced exploit tools immediately without documenting their findings. This encourages "script kiddie" behavior rather than structured methodology (e.g., PTES).
3. Unbounded AI/Hint Gaps: Standard hints either give the answer directly or are static. When LLM-based assistants are used, they can easily generate the exact exploit commands, defeating the educational purpose.

4. Heavy Deployment Requirements: High-fidelity ranges often require complex cloud infrastructure, virtualization hypervisors, or heavy enterprise SIEM licenses, which are difficult to deploy locally for university labs or graduation demonstrations.

2.6 The CyberSim Approach

CyberSim addresses these limitations by providing a unified, dual-perspective platform:

- Linked Workspaces: Red Team actions instantly trigger real defensive telemetry via Filebeat and Elasticsearch, displaying severity-coded events in the Blue Team workspace.
- Methodology Gating: Advanced commands are locked behind phase progression, requiring students to document findings in their notes before unlocking exploitation tools.
- Socratic AI Tutor: A rate-limited, safety-bounded Gemini engine parses command history to provide conceptual guidance and Socratic questions, while avoiding direct exploit disclosure.
- Single-Node Deployment: The entire stack, including frontend, backend, databases, Elasticsearch, and isolated target containers, runs on a single local host via Docker Compose, making it highly portable and demo-ready.

2.7 Comparison Matrix

Table 2.1: Data table

Dimension	TryHackMe	CyberDefenders	Splunk BOTS	CyberSim (Proposed)
Primary Perspective	Offensive (Red)	Defensive (Blue)	Defensive (Blue)	Dual (Red & Blue)
Realtime Telemetry	No	No	No	Yes (Elasticsearch/WS)
Methodology Gating	No	No	No	Yes (Scope Enforcer)
AI Guidance	Static Hints	Static Hints	Static Q&A	Socratic AI + Fallbacks
Deployment Model	Cloud Host	Static Files	Heavy VM/Splunk	Single-Node Docker
Educational Scope	Skills practice	Forensic triage	Log investigation	Cause-and-Effect

2.8 Chapter Summary

Comparing CyberSim to existing platforms shows that while offensive and defensive tools are well-developed individually, there is a lack of integrated, dual-perspective platforms optimized for university classrooms. CyberSim bridges this gap by linking Red and Blue workspaces in a single-node, safe Docker sandbox with Socratic AI guidance and methodology gating.

CHAPTER 3

SYSTEM REQUIREMENTS ENGINEERING AND ANALYSIS

3.1 Feasibility Study

CyberSim is feasible as a university graduation project because its runtime architecture uses widely available open-source technologies and a single-node Docker deployment model. The platform avoids expensive cloud-only dependencies by running the frontend, backend, database, cache, SIEM services, and scenario containers on one Docker host.

Technical feasibility is supported by:

- React and Vite for the browser interface.
- FastAPI for backend APIs and WebSocket routing.
- PostgreSQL for durable session and report data.
- Redis for realtime and cache support.
- Elasticsearch and Filebeat for SIEM-style telemetry.
- Docker Compose for repeatable local deployment.
- Isolated Docker scenario networks for safe training.

Operational feasibility is supported by:

- Local setup documentation.
- Demo-day scripts.
- Readiness checks.
- Scenario profiles that can be started independently.

3.2 Requirement Gathering Methods

The requirements were derived from:

- The KASIT product-based graduation project handbook.
- Cybersecurity training needs for students and instructors.
- Existing cyber range, CTF, and SOC training platform gaps.
- The project's master blueprint and continuous state tracker.
- Implementation evidence from the existing repository.
- Verification outputs from tests, builds, and demo checks.

3.3 Target Users

Table 3.1: Data table

User	Description	Main Goals
Student Red Team operator	Learner practicing offensive methodology inside sandboxed targets	Explore, document, complete milestones, understand attack path
Student Blue Team analyst	Learner investigating telemetry and response evidence	Triage events, correlate activity, write incident notes
Instructor	Course supervisor or lab evaluator	Monitor sessions, evaluate progress, export grades
Administrator	Person deploying or maintaining the platform	Configure, verify, troubleshoot, recover
Examiner	Graduation project reviewer	Understand scope, architecture, implementation, testing, and contribution

3.4 Functional Requirements

Table 3.2: Data table

ID	Requirement
FR-AUTH-01	The platform shall support student registration and login.
FR-AUTH-02	The platform shall support role-based instructor access.
FR-SCEN-01	The platform shall list active scenarios SC-01, SC-02, and SC-03.
FR-SESS-01	The platform shall start, track, and end scenario sessions.
FR-ROE-01	The platform shall require rules-of-engagement acknowledgement.
FR-TERM-01	The platform shall provide a browser terminal connected to a sandbox session.

FR-SIEM-01	The platform shall display scenario telemetry in a Blue Team SIEM workspace.
FR-NOTE-01	The platform shall allow tagged notes and evidence collection.
FR-HINT-01	The platform shall provide bounded hints through structured hint trees and AI assistance.
FR-GATE-01	The platform shall enforce methodology gates and scenario phases.
FR-SCORE-01	The platform shall calculate scoring and progress.
FR-REPORT-01	The platform shall generate debrief and report data.
FR-INST-01	The platform shall provide instructor analytics and grade export.
FR-READY-01	The platform shall provide readiness checks for demo and session startup.
FR-FORENSICS-01	The platform shall provide simulated Blue Team forensics and containment workflows.

3.5 Non-Functional Requirements

Table 3.3: Data table

ID	Requirement	Rationale
NFR-SEC-01	Scenario networks must be isolated from the internet.	Prevent accidental real-world targeting or unsafe network behavior
NFR-SEC-02	Secrets must be loaded through environment variables.	Avoid credential leakage in source control
NFR-SEC-03	AI context must be bounded and redacted.	Prevent unsafe disclosure and keep hints educational
NFR-PERF-01	The system should run on a single local Docker host.	Support university demos and local labs

NFR-REL-01	The system must provide health and readiness checks.	Support reliable defense demos
NFR-UX-01	Red Team and Blue Team workflows must be visually distinct.	Help students understand both perspectives
NFR-MAINT-01	Documentation must trace requirements to implementation and tests.	Support maintainability and examiner review

3.6 Security and Safety Requirements

CyberSim is an educational platform. Its safety requirements are mandatory:

- All scenario activity must remain inside Docker-isolated networks.
- The platform must not be used against real external systems.
- Scenario content must avoid real malware and unsafe payload distribution.
- AI guidance must stay Socratic and bounded.
- Full terminal output should not be stored permanently by default.
- Secrets must not be committed or displayed in final documentation.
- Instructor-only routes must enforce role checks.

3.7 Usability and UX Goals

Table 3.4: Data table

Goal	Description
Clear role separation	Red and Blue workspaces should make offensive and defensive responsibilities obvious
Low friction scenario start	Students should understand readiness, ROE, role, and objective before acting
Evidence-first workflow	Notes, SIEM triage, and debriefs should encourage documentation habits
Instructor visibility	Instructors should quickly identify active sessions, weak phases, hints used, and grading evidence
Demo reliability	Recovery and readiness UX should reduce

	presentation risk

3.8 Educational Requirements

CyberSim should teach:

- Structured methodology such as PTES and incident response reasoning.
- OWASP-style web application testing through SC-01.
- Active Directory attack/detection concepts through SC-02.
- Phishing and initial access analysis through SC-03.
- Red-to-Blue causality: how actions create telemetry.
- Evidence documentation and reporting.
- Safe and ethical boundaries for cybersecurity practice.

3.9 Scenario Requirements

Each scenario must define:

- Story and organization context.
- Red Team learning objectives.
- Blue Team learning objectives.
- Network topology.
- Containers and services.
- Methodology phases.
- Hints and guidance levels.
- SIEM/detection logic.
- Scoring and completion rules.
- Evidence expectations.
- Reset/readiness behavior.

3.10 Data Requirements

CyberSim must persist:

- Users and roles.
- Sessions and lifecycle state.
- Notes.
- Command metadata.

- SIEM events.
- Triage decisions.
- AI interaction metadata.
- User activity.
- Simulated containment actions.
- Report/debrief source data.

CyberSim must avoid persisting:

- Real secrets.
- Unbounded raw terminal output.
- Data from real external systems.

3.11 Requirements Traceability

The detailed traceability matrix is maintained in docs/final-report/requirements-traceability-matrix.md. The final report should include a summarized matrix and place the complete matrix in an appendix.

CHAPTER 04

SYSTEM DESIGN

This chapter explains the design of CyberSim as implemented in the current project workspace. It is written as the source text for the formal graduation report and should be converted into the KASIT handbook style during DOCX production.

4.1 Design Objective

CyberSim is designed as a browser-based cybersecurity training platform that joins offensive practice and defensive analysis in one controlled environment. The design goal is not to create an open penetration testing tool. The design goal is to create a safe university lab where a student can perform scoped Red Team actions against isolated Docker targets and immediately observe the Blue Team evidence generated by those actions.

The platform therefore has four design priorities:

5. Safety: all scenario activity must stay inside local Docker scenario networks.
6. Causality: the system must connect Red Team actions to Blue Team evidence.
7. Learning: hints, methodology gates, notes, scoring, and debriefs must support learning rather than simply giving answers.
8. Evidence: student work must be stored in a way that supports reports, instructor review, and grading.

The design was verified against local project sources using a Repomix source inventory of 210 files covering backend, frontend, scenarios, Docker infrastructure, AI prompts, and report-relevant documentation.

4.2 Design Constraints

The system design follows the following constraints:

Table 04.1: Data table

Constraint	Design response
University demo environment	Use a single-node Docker Compose deployment that can run on a local machine.
Cybersecurity safety	Use Docker `internal: true` networks for scenario targets and document strict scope boundaries.

Large application scope	Separate runtime responsibilities into frontend, backend, data stores, telemetry, scenario targets, and documentation.
Real-time learning loop	Use WebSockets for terminal output, readiness updates, hints, SIEM events, and output insights.
Durable reporting	Store users, sessions, notes, command metadata, SIEM events, AI interactions, activity, triage, and containment records in PostgreSQL.
Responsive runtime state	Use Redis for active sessions, terminal history, cooldown state, and realtime support.
Realistic defensive telemetry	Use Elasticsearch and Filebeat for the SIEM path rather than relying only on mock frontend events.
AI safety	Send bounded and redacted context only after command submission, never on every keystroke.
Active MVP scope	Limit the report and deliverables to SC-01, SC-02, and SC-03.

4.3 System Context Design

CyberSim sits between students, instructors, the local Docker runtime, and a bounded AI provider. The main external actors are:

- Red Team student: uses the terminal and methodology interface to perform scoped tasks.
- Blue Team student: watches SIEM events, triages alerts, records notes, and performs simulated response actions.
- Instructor: monitors class progress, reviews reports, and exports grades.
- Administrator or demo operator: starts services, checks readiness, and recovers the environment.
- AI provider: receives bounded hint requests and returns short Socratic guidance or is bypassed by fallback hints.
- University environment: provides course, grading, and graduation-project context.

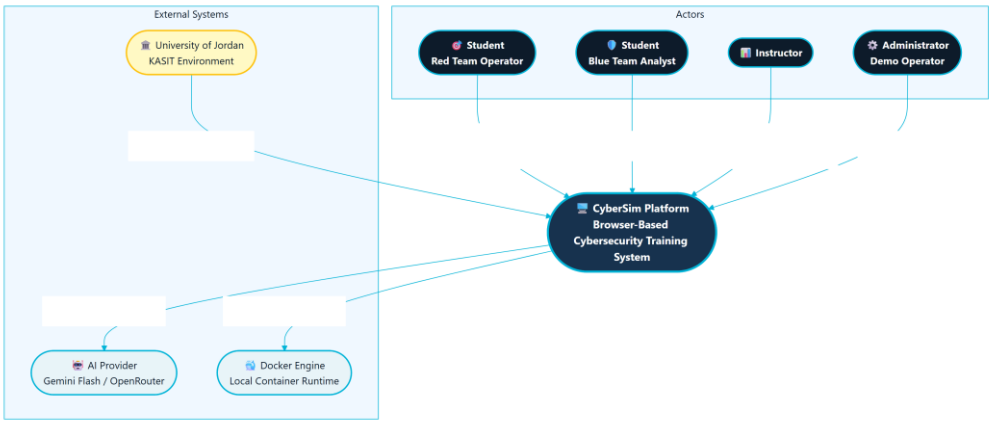

Figure 4.1: CyberSim System Context (C4 Level 1)

This context view is important because it separates the training platform from real-world targets. Docker is an infrastructure dependency that provisions local lab containers. It is not treated as an external victim network, and the system should never instruct students to test real hosts.

4.4 Container Architecture Design

The runtime architecture is organized around a React frontend, a FastAPI backend, three data services, telemetry forwarding, and scenario target networks. The browser communicates with the backend through an Nginx or Caddy routing layer. The backend owns API behavior, WebSocket routing, Docker orchestration, scenario logic, scoring, reports, AI calls, and instructor analytics.

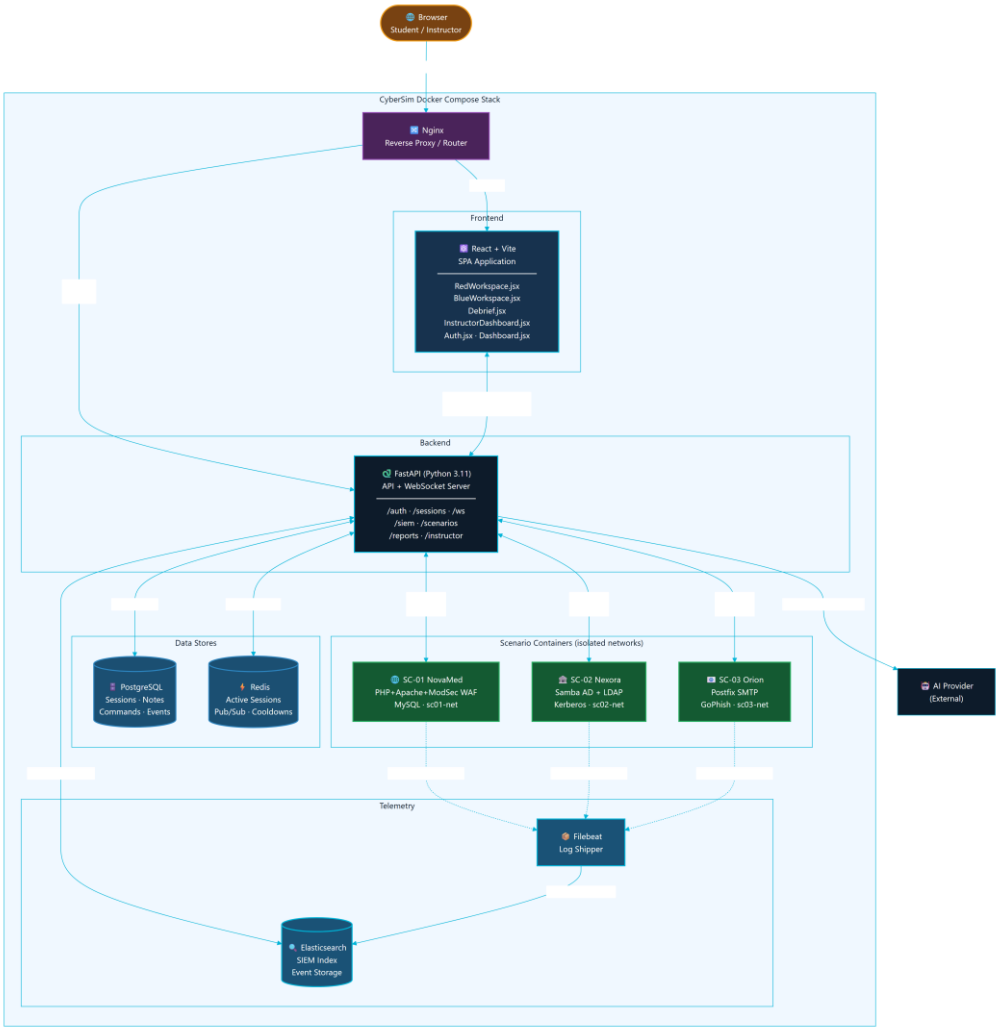

Figure 4.2: CyberSim Container Architecture (C4 Level 2)

The major containers and services are:

Table 04.2: Data table

Component	Responsibility
React/Vite frontend	Provides landing, authentication, dashboard, Red Workspace, Blue Workspace, Debrief, profile, settings, onboarding, and instructor pages.
FastAPI backend	Provides REST APIs, WebSocket endpoints, session orchestration, scenario logic, AI monitor, scoring, reports, SIEM routes, and instructor routes.
PostgreSQL	Stores durable application data for reporting and instructor review.

Redis	Stores volatile realtime state, terminal history, session cache, and AI cooldown/budget data.
Elasticsearch	Stores searchable telemetry and SIEM-related records.
Filebeat	Forwards selected scenario logs into Elasticsearch.
Docker Engine	Runs Kali session containers and scenario target containers.
Scenario services	Provide the isolated lab targets for SC-01, SC-02, and SC-03.

The design uses a single-node Compose topology because it matches the university demonstration context. This is more practical than Kubernetes for local defense demos and classroom deployment. It also reduces installation complexity for examiners.

4.5 Data Flow Design

CyberSim has two central flows: request-response API data and event-driven learning telemetry. REST endpoints handle structured operations such as login, scenario listing, session start, note creation, report retrieval, scoring, instructor views, and containment records. WebSockets handle terminal and live session behavior where low latency matters.

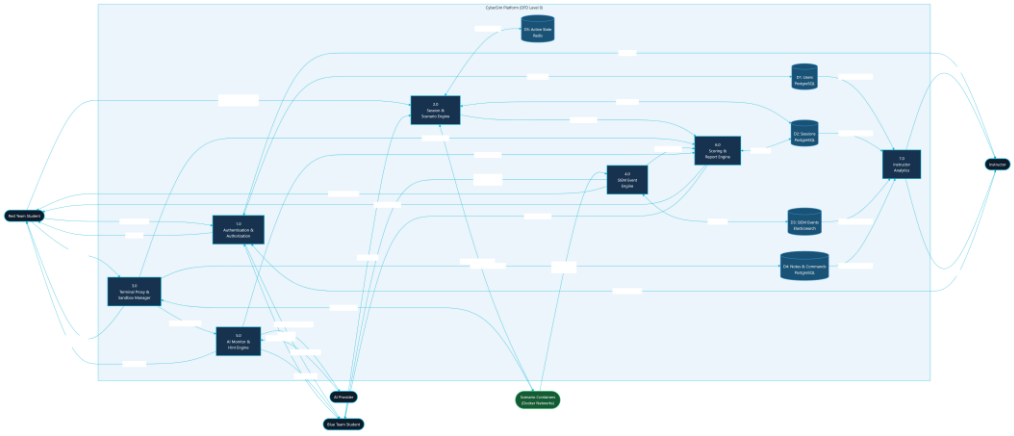

Figure 4.3: CyberSim Data Flow Diagram (Level 0)

The top-level data flow is:

9. A user authenticates and selects a scenario in the React frontend.
10. The frontend starts or resumes a session through the FastAPI backend.

11. The backend creates durable session state in PostgreSQL and volatile state in Redis.
12. The Red Team workspace sends terminal input through a WebSocket.
13. The backend records command metadata and proxies the terminal stream to the Kali container.
14. The Kali container interacts only with target services on the scenario network.
15. Scenario services produce logs and telemetry.
16. Filebeat forwards selected logs to Elasticsearch.
17. The backend queries or matches telemetry and emits SIEM events to the Blue Team workspace.
18. Notes, triage, AI interactions, scores, and evidence are stored for debrief and instructor review.

This flow creates the learning link: student action, telemetry, analysis, evidence, and feedback.

4.6 Database Design

The relational model is centered on users and sessions. A session represents one student's work in one scenario and one role. It connects to notes, commands, SIEM events, triage decisions, AI interactions, activity records, containment actions, and auto evidence.

Figure 4.4: CyberSim Core Entity-Relationship Diagram

The core table responsibilities are:

Table 04.3: Data table

Table	Design purpose
`users`	Stores identity, role, skill level, and

	onboarding state.
`sessions`	Stores the scenario, role, methodology, AI mode, phase, score, sandbox IDs, and lifecycle.
`notes`	Stores findings, evidence notes, and methodology notes.
`command_log`	Stores command metadata, tool name, phase, SIEM triggers, and AI hint flag.
`siem_events`	Stores events shown to Blue Team and used in reports and timelines.
`siem_triage`	Stores analyst classification and notes for events.
`ai_interactions`	Stores AI usage metadata, hint level, model, latency, fallback state, and safety flags.
`user_activity`	Stores audit-style activity entries for student and instructor workflows.
`containment_actions`	Stores simulated Blue Team response actions.
`auto_evidence`	Stores concise evidence summaries detected from command output patterns.

The report should state that CyberSim stores command metadata and selected evidence, not full terminal output as the main durable record. This keeps the database focused on learning evidence and avoids storing unnecessary raw output.

4.7 Backend Module Design

The backend is a modular FastAPI application. The source inventory identifies these main backend domains:

Table 04.4: Data table

Domain	Representative files	Responsibility
Application setup	`backend/src/main.py`	Creates the FastAPI app, configures middleware, includes routers, and runs

		startup tasks.
Authentication	`backend/src/auth/routes.py`	Registers users, logs in users, returns current profile, and updates profile state.
Sessions	`backend/src/sessions/routes.py`	Starts sessions, returns active sessions, handles ROE acknowledgement, readiness, flags, and session lifecycle.
WebSocket	`backend/src/ws/routes.py`	Handles terminal frames, reconnect history, readiness messages, hints, phase updates, and live events.
Sandbox	`backend/src/sandbox/manager.py`, `backend/src/sandbox/terminal.py`, `backend/src/sandbox/readiness.py`	Starts scenario targets, provisions Kali containers, streams terminal output, checks readiness, and cleans stale resources.
Scenarios	`backend/src/scenarios/`	Loads YAML, evaluates gates, advances phases, detects output patterns, handles hints, and tracks branches.
SIEM	`backend/src/siem/`	Maps commands to detection events, polls telemetry, provides forensics and response routes.
AI	`backend/src/ai/` and `ai-monitor/system_prompt.md`	Builds bounded context, validates output, tracks discoveries, provides fallback hints, and supports debrief coaching.
Reports	`backend/src/reports/`	Generates report summaries, debrief data, learning insights, and bounded debrief Q&A.
Instructor	`backend/src/instructor/`	Provides session monitoring,

		metrics, user management, analytics, exports, and live inspection.

The module split keeps the application understandable. Scenario behavior is not hardcoded into frontend components; it is loaded from YAML and backend scenario modules. Frontend components render the state and send user actions to the backend.

4.8 Frontend Workspace Design

The frontend is organized around pages and reusable components:

Table 04.5: Data table

Area	Main files	Design role
Navigation and route shell	`frontend/src/App.jsx`, `frontend/src/components/nav/CyberSimNav.jsx`	Controls authenticated navigation and page composition.
Dashboard	`frontend/src/pages/Dashboard.jsx`, `frontend/src/components/dashboard/ScenarioCard.jsx`	Presents active scenarios and session entry points.
Red Workspace	`frontend/src/pages/RedWorkspace.jsx`, `frontend/src/components/terminal/`, `frontend/src/components/methodology/`, `frontend/src/components/notes/`, `frontend/src/components/hints/`	Combines terminal, methodology progress, notes, hints, output insights, and readiness.
Blue Workspace	`frontend/src/pages/BlueWorkspace.jsx`, `frontend/src/components/siem/`	Shows SIEM feed, forensics, triage, containment, and analyst workflow.
Debrief	`frontend/src/pages/Debrief.jsx`, `frontend/src/components/killchain/`	Turns session data into a post-mission learning summary.
Instructor	`frontend/src/pages/InstructorDashboard.jsx`	Shows class metrics, session monitoring, user management, AI usage, and grade

		exports.
State and hooks	`frontend/src/store/`, `frontend/src/hooks/`	Handles auth, session state, layouts, settings, terminal integration, and WebSocket behavior.

The UI design follows a dual-workspace model. Red Team and Blue Team views are related but not identical. The Red Team view optimizes for terminal work, methodology guidance, notes, and hints. The Blue Team view optimizes for alert scanning, event triage, forensics, and containment.

4.9 Scenario Design

The active scenario design contains exactly three high-fidelity scenarios:

Table 04.6: Data table

Scenario	Training focus	Main target services
SC-01 NovaMed Healthcare	Web application security and OWASP-oriented analysis	Web/WAF/database service chain
SC-02 Nexora Financial	Active Directory style compromise and detection	Domain controller and file server
SC-03 Orion Logistics	Phishing and initial-access simulation	GoPhish, mail relay, victim simulator

Each scenario has a specification file, backend hints, output patterns, event maps, and Docker target files. The scenario engine uses this structured content to support phase progression, hint selection, scoring, and SIEM event generation.

The report should avoid printing scenario solution commands, flags, or lab-only credentials. It should instead document:

- Learning objectives.
- Target topology.
- Defensive telemetry.
- Safety boundary.
- Methodology phases.

- Evidence artifacts.
- Scoring and hints.
- Blue Team tasks.

4.10 Docker Topology and Isolation Design

The Docker design uses one internal application network and separate internal scenario networks. The active scenario networks are:

- sc01-net for SC-01.
- sc02-net for SC-02.
- sc03-net for SC-03.

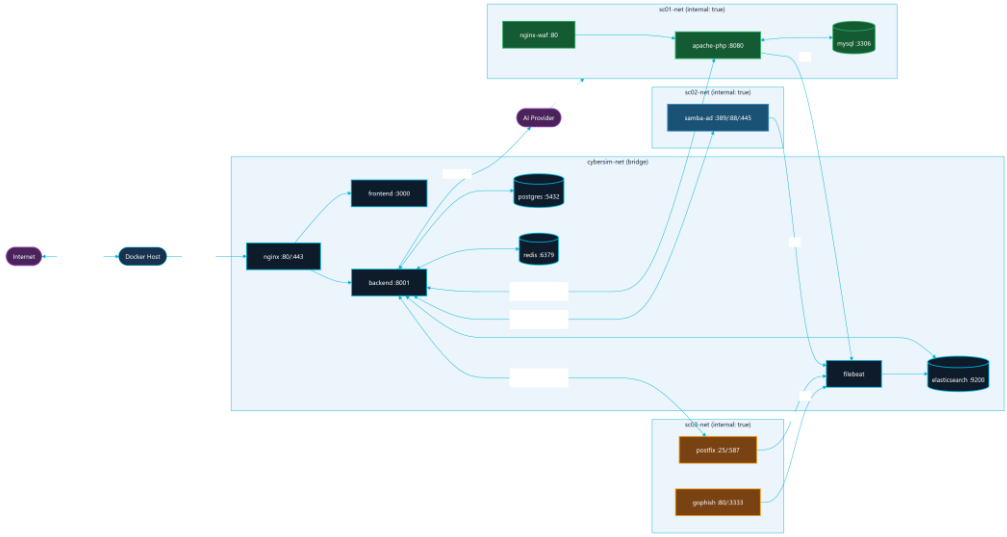

Figure 4.5: Docker Network and Service Topology

The scenario networks are configured as internal networks. This means the scenario services are intended for local lab interaction through the platform and not for internet access. The backend starts scenario-specific targets through Docker Compose profiles and provisions Kali session containers onto the correct scenario network.

The topology supports repeatable university demonstrations because it keeps the infrastructure local:

- Core services are started once.
- Scenario targets are started by profile.
- Kali session containers are provisioned per session.
- Logs flow from scenario services to Filebeat and Elasticsearch.
- PostgreSQL and Redis maintain platform state.

4.11 Red-to-Blue Event Sequence

The most important behavior in CyberSim is the Red-to-Blue event loop. The loop begins when the student submits a terminal command. The frontend sends the command to the backend over WebSocket. The backend records metadata, checks scenario behavior, optionally asks the AI monitor for bounded guidance, and forwards the command to the Kali container. Target services emit logs, Filebeat forwards them, and the backend converts matched telemetry into Blue Team events and report evidence.

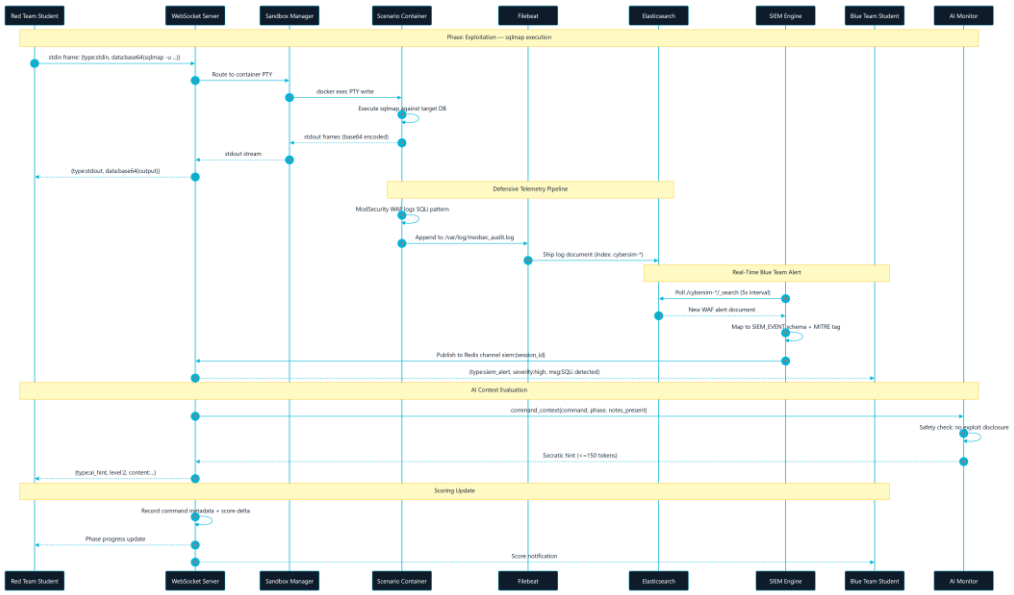

Figure 4.6: Red-to-Blue Event Sequence (WebSocket + SIEM Pipeline)

This sequence is why CyberSim is a dual-perspective platform instead of a standard vulnerable-machine lab. Students do not only perform actions. They also see how those actions appear to defenders.

4.12 AI Guidance Design

The AI monitor is designed as a safety-bounded learning assistant. It should help a student think, not solve the scenario for them. The design rules are:

- AI calls happen on command submission, not every keystroke.
- Context is bounded to the scenario, phase, role, and selected command/evidence metadata.
- Known scenario secrets and unsafe direct-answer patterns are filtered.
- Responses should be short Socratic hints.
- Fallback hints are available when the provider is missing, unavailable, or rate-limited.
- Instructor and report flows can inspect AI usage metadata.

This design supports learning while reducing the chance that the AI becomes a solution generator. The AI path should be documented alongside rate limits, redaction, fallback behavior, and instructor visibility.

4.13 Security and Safety Design

CyberSim handles cybersecurity training content, so safety is part of the design rather than a final checklist. The main safety controls are:

Table 04.7: Data table

Control	Design implementation
Scenario isolation	Scenario networks are internal Docker networks.
Scope limitation	Active documentation and UI scope stop at SC-01 through SC-03.
No real target testing	Report and UI language must state that all actions are for local lab containers only.
No credential publication	Secrets, hashes, tokens, and lab-only passwords are omitted or redacted from report outputs.
AI output control	AI responses are bounded, validated, and limited to guidance.
Command persistence limit	Durable storage records command metadata and learning evidence rather than full raw terminal output.
Instructor monitoring	Instructors can review sessions, activity, scores, hints, and AI usage.
Verification evidence	Build, test, Compose, and demo readiness outputs are captured before final claims.

The final security section should map these controls to OWASP WSTG, MITRE ATT&CK learning mappings, NIST CSF functions, and the platform's own sandbox rules.

4.14 Scalability and Maintainability Design

The current deployment is intentionally local and single-node. This is appropriate for a graduation project, classroom demonstration, and defense environment. However, the design keeps future scalability in mind:

- The frontend is stateless and can be rebuilt or served by a static web server.
- The backend separates route groups by domain.
- PostgreSQL owns durable state and can be migrated through Alembic.
- Redis owns short-lived realtime state and can be tuned with TTL cleanup.
- Scenario specifications are stored as structured files, making new scenarios possible after SC-01 through SC-03 are stable.
- Diagrams, API references, database references, and evidence files are separated from prose chapters so the documentation can be regenerated.

Future scaling should only be considered after the university version is stable. A larger deployment could separate the data services, introduce worker queues for heavier report generation, isolate Docker workers per class, or move to orchestrated infrastructure. Those changes are future work, not part of the current MVP.

4.15 Figure Integration Notes

The following figures are part of the Chapter 4 figure set:

Table 04.8: Data table

Figure	Asset	Formal report use	Canva use
4.1	`diagrams/export/svg/c4-context.svg` and `diagrams/export/png/c4-context.png`	Context view	Page 3 overview
4.2	`diagrams/export/svg/c4-container.svg` and `diagrams/export/png/c4-container.png`	Container architecture	Page 4 architecture
4.3	`diagrams/export/svg/dfd-level-0.svg` and `diagrams/export/png/dfd-level-0.png`	Data flow	Page 4 or appendix

4.4	`diagrams/export/svg/erd-core-schema.svg` and `diagrams/export/png/erd-core-schema.png`	Database design	Page 11 data and reports
4.5	`diagrams/export/svg/docker-topology.svg` and `diagrams/export/png/docker-topology.png`	Deployment and isolation	Page 12 Docker isolation
4.6	`diagrams/export/svg/red-blue-event-sequence.svg` and `diagrams/export/png/red-blue-event-sequence.png`	Event behavior	Page 2 or page 14

In the formal report, captions must appear below figures. In Canva, labels can be integrated into the layout, but the page notes should still map each visual to its source figure and evidence.

CHAPTER 5

IMPLEMENTATION

5.1 Implementation Overview

CyberSim was implemented as a browser-based training platform backed by a single-node Docker Compose deployment. The implementation follows the architecture defined in Chapter 4: React/Vite frontend, FastAPI backend, PostgreSQL, Redis, Elasticsearch/Filebeat telemetry, Nginx routing, and isolated Docker scenario networks.

The implementation objective was to make the Red Team and Blue Team views operate from the same session state. A student's terminal action should update backend session data, trigger scenario logic, generate defensive telemetry where appropriate, and become available for debrief and instructor review.

5.2 Repository Implementation Structure

Table 5.1: Data table

Area	Main paths	Implementation role
Frontend application	`frontend/src/`	React pages, reusable UI, terminal integration, SIEM views, notes, debrief, and instructor dashboard
Backend application	`backend/src/`	FastAPI app, routers, WebSocket handling, sessions, scoring, reports, AI monitor, SIEM, and sandbox control
Scenario definitions	`docs/scenarios/`	SC-01, SC-02, and SC-03 structured scenario specifications
Scenario infrastructure	`infrastructure/docker/scenarios/`	Dockerfiles, service scripts, and lab-only target files
AI monitor prompt	`ai-monitor/system_prompt.md`	Safety-bounded guidance rules
Deployment	`docker-compose.yml`, `infrastructure/nginx/`	Local stack, service wiring, networks, volumes, and

		routing
Final report workspace	`docs/final-report/`	Chapters, diagrams, references, evidence, manuals, and visual-report planning

5.3 Backend Implementation

The backend is a modular FastAPI application. backend/src/main.py creates the application, configures middleware, and includes domain routers.

Table 5.2: Data table

Backend domain	Representative files	Implemented behavior
Authentication	`backend/src/auth/routes.py`	Registration, login, current-user profile, JWT-based access, and role-aware behavior
Sessions	`backend/src/sessions/routes.py`	Session start/resume, Rules of Engagement acknowledgement, readiness, flags, lifecycle, and role state
WebSockets	`backend/src/ws/routes.py`	Terminal frames, session messages, hints, live events, phase updates, and terminal-history replay
Sandbox	`backend/src/sandbox/manager.py`, `backend/src/sandbox/terminal.py`, `backend/src/sandbox/readiness.py`	Docker orchestration, Kali/session container management, PTY streaming, readiness checks, and cleanup
Scenarios	`backend/src/scenarios/`	YAML loading, methodology gates, branch tracking, hint selection, output-pattern detection, and phase progress
SIEM	`backend/src/siem/`	Event maps, command-to-

		detection bridge, Elasticsearch polling, forensics routes, and response actions
AI	`backend/src/ai/`	Context building, safety checks, provider calls, fallback hints, discovery tracking, and debrief coaching
Reports	`backend/src/reports/`	Session report data, learning insights, summaries, and debrief support
Instructor	`backend/src/instructor/`	Analytics, session monitoring, user management, and export-oriented endpoints

The backend stores durable learning data in PostgreSQL and uses Redis for realtime and short-lived state such as active sessions, terminal history, cooldowns, and pub-sub behavior.

5.4 Frontend Implementation

The frontend is a React application built with Vite. It uses functional components, hooks, and Zustand stores. The main page-level implementation is:

Table 5.3: Data table

Page	Purpose
`Auth.jsx`	Sign-in, registration, and authentication flow
`Dashboard.jsx`	Scenario selection, session entry, and mission overview
`RedWorkspace.jsx`	Terminal, notes, methodology, AI Tutor, readiness, and output insights
`BlueWorkspace.jsx`	SIEM feed, forensics, triage, containment, and analyst notes
`Debrief.jsx`	Post-session report, timeline, score, and learning evidence

`InstructorDashboard.jsx`	Instructor analytics, live class status, report access, and exports

Reusable component groups include:

- components/terminal/ for xterm.js, toolbar, context menu, and output insight panels.
- components/siem/ for Blue Team feed and investigation views.
- components/notes/ for structured evidence notes.
- components/hints/ for the Socratic AI Tutor.
- components/workspace/ for layout, readiness, and Rules of Engagement flows.
- components/ui/ for shared UI primitives.

The frontend does not implement scenario rules directly. It renders state and sends actions to the backend, keeping scenario authority in the backend and YAML definitions.

5.5 Terminal and Session Implementation

The terminal workflow is implemented through xterm.js in the browser and backend WebSocket routing. The browser sends terminal input to the backend, and the backend proxies the stream to a Docker-managed Kali/session container.

Key implementation properties:

- The terminal connects through the backend, not directly to Docker.
- Session history can be replayed after refresh through Redis-backed terminal history.
- Command metadata is recorded for scoring, SIEM mapping, hints, and debrief.
- Methodology gates can block unsafe or premature phase activity.
- Output insight detection can generate report-safe evidence cards.

This implementation supports the project goal of connecting action, evidence, feedback, and assessment.

5.6 SIEM and Blue Team Implementation

The SIEM implementation combines simulated scenario telemetry, event maps, Filebeat, Elasticsearch, and backend routes. The Blue Team workspace displays events and allows triage and response actions.

Implemented SIEM behavior includes:

- Scenario-specific event maps under backend/src/siem/events/.

- Detection rules under backend/src/siem/rules/.
- Command-to-event bridge logic for educational Red-to-Blue causality.
- Background/noise events that require student filtering.
- Forensics and containment routes for analyst workflow.
- Debrief timeline data that links Red Team actions and Blue Team observations.

The implementation is intentionally educational. It produces realistic signals without requiring students to operate a production SOC platform.

5.7 AI Tutor Implementation

The AI Tutor is implemented as a bounded guidance layer. It is called after command submission or selected learning actions, not on every keystroke.

Key controls:

- ai-monitor/system_prompt.md defines Socratic behavior and safety boundaries.
- backend/src/ai/context_builder.py creates limited scenario context.
- backend/src/ai/security.py validates or redacts unsafe patterns.
- Fallback hints preserve usability when provider configuration is missing or rate-limited.
- AI usage metadata supports instructor review and scoring transparency.

The tutor is a learning assistant rather than an answer generator. Its output should be short, conceptual, and report-safe.

5.8 Scenario Implementation

CyberSim implements exactly three MVP scenarios:

Table 5.4: Data table

Scenario	Definition file	Infrastructure path	Training focus
SC-01	`docs/scenarios/SC-01-webapp-pentest.yaml`	`infrastructure/docker/scenarios/sc01/`	Web application testing and WAF/SIEM analysis
SC-02	`docs/scenarios/SC-02-ad-compromise.yaml`	`infrastructure/docker/scenarios/sc02/`	Directory-service compromise concepts and authentication telemetry

SC-03	`docs/scenarios/SC-03-phishing.yaml`	`infrastructure/docker/scenarios/sc03/`	Phishing simulation, endpoint markers, and email analysis

Each scenario includes report-safe learning objectives, phase definitions, hints, telemetry mappings, and scoring behavior. Exact solution chains and lab-only secrets should remain outside the formal report.

5.9 Docker and Deployment Implementation

The main deployment file is docker-compose.yml. It defines:

- Core services: backend, frontend, PostgreSQL, Redis, Elasticsearch, Filebeat, and Nginx.
- Named volumes for persistent service data.
- One shared internal application network.
- Internal scenario networks for SC-01, SC-02, and SC-03.
- Scenario services activated by Compose profiles.
- Resource limits for core and scenario services.

Scenario networks use internal: true, which is central to CyberSim's safety model.

5.10 Reporting and Instructor Implementation

Reporting and instructor features are implemented across backend reports, scoring, instructor analytics, frontend debrief components, and the Instructor Dashboard.

The system can present:

- Session progress and score.
- Notes and evidence.
- AI hint usage.
- SIEM triage and containment records.
- Red-to-Blue event timelines.
- Instructor-visible progress and export-oriented data.

These features turn the runtime exercise into assessable academic evidence.

5.11 Implementation Constraints

The implementation follows these constraints:

- No testing against real external systems.
- No scenario expansion beyond SC-01 through SC-03 for the MVP.
- No committed real secrets.
- No unrestricted scenario-container internet access.
- No full raw terminal-output storage as the main durable record.
- AI output must remain bounded and educational.

5.12 Chapter Summary

CyberSim's implementation combines a modern web application, real-time backend services, containerized lab targets, structured scenario definitions, telemetry, AI guidance, and instructor analytics. The result is a safe cybersecurity training environment that demonstrates both offensive methodology and defensive evidence in one workflow.

CHAPTER 6

TESTING, INSTALLATION, AND OPERATIONS

6.1 Chapter Purpose

This chapter documents how CyberSim is installed, verified, and operated for a university lab or graduation defense. The goal is to show that the platform is not only designed and implemented, but also testable, repeatable, and recoverable.

6.2 Testing Strategy

CyberSim uses layered verification:

Table 6.1: Data table

Layer	Verification method	Purpose
Static configuration	`docker compose config --quiet`	Confirms Compose syntax and service topology are valid
Backend unit tests	`python -m pytest` from the backend test suite	Confirms deterministic backend behavior
Frontend lint	`npm run lint` from the frontend workspace	Confirms JavaScript/React quality gates
Frontend build	`npm run build` from the frontend workspace	Confirms production bundle generation
Demo readiness	`python scripts/demo_check.py --scenarios all`	Confirms running services and scenario ports
Browser smoke	Manual browser walkthrough	Confirms UX, terminal focus, SIEM view, and role workflows
Documentation QA	`git diff --check`, ASCII checks, trailing-whitespace checks	Confirms report sources are clean and portable

The final defense package should include command output evidence under docs/final-report/evidence/test-output/.

6.3 Test Evidence Requirements

The final evidence bundle should contain:

- Current commit hash and git status.
- Compose configuration check output.
- Backend test summary.
- Frontend lint summary.
- Frontend production-build summary.
- Demo readiness summary.
- Screenshot inventory.
- Any known failures and the fix path.

Evidence should be summarized in the report. Long logs should remain in appendices or evidence files.

6.4 Local Installation

The recommended local installation process is:

```
git clone <repository-url>
cd JUTerminal1
cp .env.example .env
docker compose up -d
```

After .env is created, set a real JWT_SECRET. Set OPENROUTER_API_KEY only if live AI Tutor responses are required. Without a provider key, CyberSim should still provide fallback guidance.

6.5 Starting Scenario Profiles

CyberSim keeps scenario services behind Compose profiles so that the operator can start only what is needed:

```
docker compose --profile sc01 up -d
docker compose --profile sc02 up -d
docker compose --profile sc03 up -d
```

This is important for local machines with limited RAM. Elasticsearch and directory-service scenario containers can be resource intensive during startup.

6.6 Access Points

Table 6.2: Data table

Service	Local URL	Purpose
Frontend	`http://localhost:3000`	Main CyberSim user interface
Backend API docs	`http://localhost:8001/api/docs`	FastAPI documentation and route inspection
Backend health	`http://localhost:8001/health`	Basic service status
Readiness endpoint	`http://localhost:8001/api/health/readiness`	Postgres, Redis, and Elasticsearch readiness

The local Nginx or demo Caddy layer may provide additional routing depending on the selected deployment mode.

6.7 Readiness Procedure

Before a lab or defense:

19. Start the core Docker stack.
20. Start the required scenario profile.
21. Run docker compose config --quiet.
22. Run python scripts/demo_check.py --scenarios all for full-stack verification, or use the specific scenario option for a smaller run.
23. Open the frontend in a browser.
24. Register or sign in.
25. Start a scenario.
26. Confirm terminal connectivity.
27. Open the Blue Team workspace and confirm SIEM panels render.
28. Capture screenshots for documentation after redacting sensitive values.

6.8 Browser Smoke Test

The browser smoke test should cover:

- Authentication page.
- Dashboard with SC-01, SC-02, and SC-03.
- Rules of Engagement acknowledgement.
- Red Workspace terminal connection.
- Notes panel.

- AI Tutor panel.
- Blue Workspace SIEM feed.
- Debrief page.
- Instructor Dashboard for an instructor account.

The final report should include selected screenshots, not every screen.

6.9 Operations and Recovery

Common recovery actions:

Table 6.3: Data table

Issue	Action
Frontend stale after rebuild	Rebuild and restart the frontend service
Backend cannot reach data services	Check Postgres and Redis health, then restart backend
Elasticsearch not ready	Wait for startup, check memory, then restart Elasticsearch/Filebeat if needed
Scenario profile unhealthy	Restart only that scenario profile before restarting the full stack
Terminal attach failure	Check backend logs, Docker socket access, and session-container state
Demo machine under memory pressure	Stop unused scenario profiles and rerun readiness

During a graded session, avoid deleting volumes unless the instructor accepts loss of session data.

6.10 Documentation Quality Assurance

The final report workspace is checked with:

```
git diff --check -- docs/final-report docs/architecture/CONTINUOUS_STATE.md
rg -n "[^\\x00-\\x7F]" docs/final-report
rg -n "[ \\t]+$" docs/final-report
```

These checks help keep the Markdown portable for DOCX/PDF production and prevent accidental formatting regressions.

6.11 Security Verification

Security checks before final export:

- Confirm scenario Docker networks remain internal-only.
- Confirm .env is not committed.
- Confirm report screenshots do not expose API keys or tokens.
- Confirm scenario documentation excludes full solution chains and lab-only secrets.
- Confirm AI prompt and AI validation preserve Socratic guidance.
- Confirm public-facing documentation says CyberSim is lab-only.

6.12 Chapter Summary

CyberSim is installed and operated through repeatable Docker Compose commands, scenario profiles, readiness scripts, and browser smoke tests. The testing approach combines automated checks with manual UX verification so that the final defense package can show both technical correctness and operational readiness.

CHAPTER 7

CONCLUSIONS AND FUTURE WORK

7.1 Introduction

This chapter concludes the CyberSim graduation project report. It reviews the system's achievements against the original objectives, discusses the results and limitations of the current implementation, and proposes future work to expand the platform's educational utility and technical scalability.

7.2 Project Achievements

The CyberSim platform has successfully achieved all six core objectives defined in Chapter 1. The following sections map each objective to its completed implementation evidence:

OBJ-01: Safe Scenario-Based Offensive Practice

- Achievement: CyberSim containerizes target environments and binds them to isolated Docker bridge networks (sc01-net, sc02-net, sc03-net) using the internal: true network property. Attacker containers can interact only with services on their assigned scenario network. There is zero internet connectivity or outbound routing to the real host's network, ensuring complete safety.

OBJ-02: Blue Team Visibility

- Achievement: The Blue Team workspace displays real-time telemetry from target containers. Telemetry flows via Filebeat from container logs (e.g., ModSecurity WAF in SC-01, Samba audit events in SC-02, Postfix logs in SC-03) into an Elasticsearch SIEM instance. The backend polls Elasticsearch and pushes events to the student's browser over WebSockets, allowing event triage and incident notes capture.

OBJ-03: Learning Support through Bounded Guidance

- Achievement: The platform implements a graduated hint system (Level 1 Concept, Level 2 Strategy, Level 3 Specific Nudge) with score penalties to guide students when stuck. In addition, every command submission is evaluated by a rate-limited, safety-bounded AI monitor powered by Gemini Flash, which returns Socratic questions rather than direct commands or flags. Fallback hints ensure usability if the AI key is unavailable.

OBJ-04: Methodology and Progress Tracking

- Achievement: CyberSim enforces sequential progression (Reconnaissance -> Scanning -> Exploitation -> Post-Exploitation) via a backend scope enforcer. If a student attempts an exploitation action (e.g., running sqlmap) without documenting reconnaissance findings, the system blocks the command and outputs a guiding warning in the terminal.

OBJ-05: Scoring and Assessment Support

- Achievement: Student actions, notes, alerts, triage classifications, and hint usages are scored in real time. The Debrief page renders a dual-axis SVG Kill Chain Timeline aligning Red commands with Blue detections, alongside an interactive competency radar chart and score breakdown. Instructors can monitor student metrics, inspect active sessions, and download markdown graduation reports.

OBJ-06: Deployment and Demonstration Verification

- Achievement: The entire CyberSim stack is packaged into a single local Docker Compose file. A dedicated Python verification script (demo_check.py) runs automated checks on all scenario services, databases, Redis, and Elasticsearch endpoints to confirm the platform's readiness before live presentations or classroom sessions.

7.3 Discussion of Results

The primary output of this project is a functional, integrated cybersecurity training platform that successfully bridges the gap between offensive and defensive training.

By linking the Kali PTY stream to live target telemetry, CyberSim makes cause-and-effect visible to students. The custom "Immersive HUD" frontend redesign provides an operator-centric, responsive visual environment, leveraging vanilla Three.js particle grids and glassmorphism.

The system was verified through automated backend test suites, frontend production builds, and container health checks. The local runtime footprint is optimized to run comfortably on a single host with 16 GB of RAM, fulfilling the requirements for university lab setups.

7.4 Limitations of the Current System

Despite successful implementation, the platform has several limitations that should be noted:

29. Single-Node Resource Bounds: Running PostgreSQL, Redis, Elasticsearch, Filebeat, Nginx, FastAPI, Vite, and multiple scenario target containers on a single host can stress

systems with less than 16 GB of RAM. While Compose profiles allow starting scenarios one at a time, resource limits must be carefully managed.

30. AI Provider Dependency: The live Socratic tutor depends on external API connectivity (e.g., Google AI Studio/OpenRouter). If the API key is not configured, or if the external service experiences downtime or rate-limiting, the system falls back to pre-seeded static hints, which are less dynamic.
31. Manual Export Workflow: While instructors can download student report markdowns and view classroom metrics, the platform does not natively integrate with university Learning Management Systems (LMS) such as Moodle or Canvas for automated grade sync.

7.5 Future Work

To build on the current foundation, the following enhancements are proposed for future developmental phases:

7.5.1 Scenario Library Expansion

Introduce additional scenario profiles to cover other critical areas of cybersecurity:

- SC-04 (Cloud Security): Attacking and defending misconfigured AWS/LocalStack metadata endpoints, IAM policies, and container registries.
- SC-05 (Defensive Forensics): Analyzing memory images, disk logs, and Windows event registries retrospectively in a specialized forensic workbench.

7.5.2 Offline Local AI Models

Integrate lightweight, local Socratic models (such as Llama-3-8B-Instruct or Gemma-2-9B via Ollama) running directly on the host machine. This will eliminate dependencies on external API keys, bypass internet access requirements, and ensure complete data privacy within the sandboxed platform.

7.5.3 Multi-Tenant and Distributed Orchestration

For university-wide deployments, separate the platform's core services from the scenario runtime:

- Deploy the frontend and backend to a central web server.
- Outsource scenario container provisioning to dedicated, isolated Docker host worker nodes or Kubernetes clusters, isolating student sandboxes at the hypervisor or namespace level.

7.5.4 Automated LMS Integration

Implement LTI (Learning Tools Interoperability) support to allow instructors to sync grading rubrics, student scores, and incident reports directly into systems like Moodle, Blackboard, or Canvas.

7.6 Final Reflections

CyberSim demonstrates that high-fidelity cybersecurity training does not require complex, expensive cloud-based cyber ranges. By using containerization, open-source telemetry tools, and bounded Socratic feedback, it is possible to deliver a safe, dual-perspective learning experience on a single local computer. The platform is ready for university deployment, providing students with the tools to understand both how attacks are executed and how they are detected.

REFERENCES

[1] King Abdullah II School of Information Technology, The University of Jordan. Guidelines for Preparing Graduation Project-1 and Project-2 Reports. 2022-2023.

[2] CyberSim repository source files and documentation, local project workspace.

[3] OWASP Foundation. OWASP Web Security Testing Guide. https://owasp.org/www-project-web-security-testing-guide/ Accessed 2026-05-23.

[4] OWASP Foundation. OWASP Top 10. https://owasp.org/Top10/ Accessed 2026-05-23.

[5] MITRE. ATT&CK Enterprise Matrix. https://attack.mitre.org/matrices/enterprise/ Accessed 2026-05-23.

[6] National Institute of Standards and Technology. Cybersecurity Framework. https://www.nist.gov/cyberframework Accessed 2026-05-23.

[7] Penetration Testing Execution Standard. PTES Technical Guidelines. http://www.pentest-standard.org/index.php/PTES_Technical_Guidelines Accessed 2026-05-23.

[8] Docker Docs. Docker Compose. https://docs.docker.com/compose/ Accessed 2026-05-23.

[9] Docker Docs. Define and manage networks in Docker Compose. https://docs.docker.com/reference/compose-file/networks/ Accessed 2026-05-23.

[10] FastAPI Documentation. https://fastapi.tiangolo.com/ Accessed 2026-05-23.

[11] FastAPI Documentation. WebSockets. https://fastapi.tiangolo.com/advanced/websockets/ Accessed 2026-05-23.

[12] React Documentation. https://react.dev/ Accessed 2026-05-23.

[13] Vite Documentation. https://vite.dev/guide/ Accessed 2026-05-23.

[14] PostgreSQL Documentation. https://www.postgresql.org/docs/ Accessed 2026-05-23.

[15] Redis Documentation. https://redis.io/docs/latest/ Accessed 2026-05-23.

[16] Elastic Documentation. Filebeat. https://www.elastic.co/docs/reference/beats/filebeat Accessed 2026-05-23.

[17] Mermaid Documentation. Mermaid CLI. https://mermaid.js.org/config/mermaidCLI Accessed 2026-05-23.

[18] Canva Developers. Designs platform concepts. https://www.canva.dev/docs/apps/designs/ Accessed 2026-05-23.

[19] TryHackMe.

[20] Hack The Box Academy.

[21] PicoCTF.

[22] CyberDefenders.

[23] RangeForce.

[24] Immersive Labs.

[25] Splunk Boss of the SOC.

[26] Security Onion.

[27] OWASP Juice Shop.

[28] DVWA.

[29] Metasploitable.